САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бекэнд-разработка

Отчёт

Лабораторная работа №4 «Развёртывание приложения на удалённом сервере»

Выполнил:

Клименко Богдан Владимирович

Группа БР 1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Развернуть микросервисное приложение фото-соцсети (созданное в ЛР3) на удалённом сервере:

- провести предварительную настройку сервера (установка зависимостей);
- развернуть приложение;
- настроить nginx в режиме обратного прокси для доступа извне.

Ход работы

1. Подготовка среды

В качестве удалённого сервера использовано изолированное Linux-окружение WSL2 (Windows Subsystem for Linux) с Ubuntu 22.04. Это полноценный Linux с собственным ядром, сетью и файловой системой, работающий отдельно от Windows-хоста. По функциональности он эквивалентен удалённому серверу с Ubuntu: те же команды, тот же Docker, тот же nginx.

Установка WSL2 и Ubuntu:

```
wsl –install
```

После установки создан пользователь с правами sudo.

2. Установка Docker Engine

Внутри Ubuntu установлен Docker Engine из официального репозитория:

```
# Обновление и установка утилит
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg lsb-release

# GPG-ключ Docker
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

# Репозиторий Docker
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu \
```

```
  $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Установка
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

# Пользователь в группу docker
sudo usermod -aG docker $USER
```

Дополнительно настроено:

- /etc/docker/daemon.json с DNS-серверами 8.8.8.8, 1.1.1.1 — из-за нестабильного DNS WSL2, иначе Docker не мог скачать образы с Docker Hub.
- /etc/resolv.conf с теми же DNS и флаг chattr +i для защиты от перезаписи WSL2.
- iptables-legacy вместо nftables — для корректной работы сети Docker в WSL2.

3. Развёртывание приложения

Проект скопирован в домашнюю директорию Linux-файловой системы (не в /mnt/c/..., чтобы избежать проблем с производительностью и правами доступа):

```
rsync -av --exclude='node_modules' --exclude='dist' --exclude='*.sqlite' \
  /mnt/c/Projects/backend-lab1/services/ ~/services/
```

Создан файл **.env** с секретами:

```
JWT_SECRET=prod_jwt_secret_...
SERVICE_TOKEN=prod_service_token_...
RABBITMQ_DEFAULT_USER=admin
RABBITMQ_DEFAULT_PASS=rabbitmq_secure_password_2026
```

4. Адаптация Dockerfile для production

Исходные Dockerfile из ЛР3 использовали node:20-slim и пытались скачать prebuilt-бинарь libvips с GitHub при установке sharp. Из-за нестабильной сети GitHub это приводило к таймаутам.

Dockerfile переработаны на node:20-alpine с системным vips-dev и py3-setuptools (последний нужен, потому что Python 3.12 удалил модуль distutils, используемый node-gyp):

```
# ---------- Stage 1: build ----------
FROM node:20-alpine AS builder

WORKDIR /app

RUN apk add --no-cache python3 py3-setuptools make g++ vips-dev

ENV npm_config_build_from_source=true

COPY package*.json ./
RUN npm ci

COPY tsconfig.json ./
COPY src ./src

RUN npm run build
RUN npm prune --production

# ---------- Stage 2: runtime ----------
FROM node:20-alpine

WORKDIR /app

RUN apk add --no-cache vips-dev

COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/dist ./dist
COPY package*.json ./

RUN mkdir -p temp-uploads

EXPOSE 3001
CMD ["node", "dist/index.js"]
```

Ключевые моменты:

- vips-dev **и в builder, и в runtime** — sharp требует libvips-cpp.so.42, которого нет в пакете vips.
- npm_config_build_from_source=true — форсирует сборку sharp из исходников с использованием системного libvips, минуя попытки скачать бинарник с GitHub.
- npm prune --production в builder и **копирование готовых** node_modules в runtime — чтобы не пересобирать sharp во второй стадии.

Dockerfile для post-service, comment-service и social-service идентичны, отличаются только строкой EXPOSE.

5. Сервис Nginx в docker-compose

В docker-compose.yml добавлен пятый сервис — **nginx**:

```
nginx:
  image: nginx:1.25-alpine
  container_name: nginx
  ports:
    - "80:80"
  volumes:
    - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    - ./nginx/conf.d:/etc/nginx/conf.d:ro
    - uploads-data:/var/www/uploads:ro
  networks:
    - backend
  depends_on:
    - auth-service
    - post-service
    - comment-service
    - social-service
  restart: always
```

Также из всех сервисов убраны строки ports: (3001–3004) — теперь эти порты доступны только внутри docker-сети, а извне — только через nginx на порту 80.

6. Конфигурация Nginx

nginx/nginx.conf — базовый конфиг с включением /etc/nginx/conf.d/*.conf и client_max_body_size 25M для загрузки фото.

nginx/conf.d/default.conf — маршрутизация по путям:

```
server {
    listen 80;
    server_name _;

    resolver 127.0.0.11 valid=10s ipv6=off;

    location /api/auth {
        set $upstream "auth-service:3001";
        rewrite ^/api/auth/?(.*)$ /auth/$1 break;
        proxy_pass http://$upstream;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /api/users {
        set $upstream "auth-service:3001";
        rewrite ^/api/users/?(.*)$ /users/$1 break;
        proxy_pass http://$upstream;
        # ...
    }

    location /api/posts {
        set $upstream "post-service:3002";
        rewrite ^/api/posts/?(.*)$ /posts/$1 break;
        proxy_pass http://$upstream;
        # ...
    }
```

```
    location /api/comments {
        set $upstream "comment-service:3003";
        rewrite ^/api/comments/?(.*)$ /comments/$1 break;
        proxy_pass http://$upstream;
        # ...
    }

    location /api/social {
        set $upstream "social-service:3004";
        rewrite ^/api/social/?(.*)$ /social/$1 break;
        proxy_pass http://$upstream;
        # ...
    }

    location /uploads/ {
        alias /var/www/uploads/;
        expires 7d;
        add_header Cache-Control "public, immutable";
    }

    location / {
        return 200 'Nginx is running for PhotoSocial API';
        add_header Content-Type text/plain;
    }
}
```

Технические решения и почему они такие:

1. resolver 127.0.0.11 valid=10s ipv6=off; — используем встроенный DNS Docker. Это позволяет nginx резолвить имена контейнеров **при каждом запросе**, а не только при загрузке конфига. Без этого nginx падал с host not found in upstream "auth-service", если сервис ещё не был готов к моменту старта nginx.
2. set $upstream "auth-service:3001"; перед proxy_pass — обязательное условие для работы resolver в динамическом режиме. Если написать proxy_pass http://auth-service:3001/; напрямую, nginx резолвит хост **один раз при загрузке конфига**.
3. rewrite ^/api/auth/?(.*)$ /auth/$1 break; — отсекает префикс /api/auth и преобразует его в /auth. Без этого nginx передал бы на сервис путь

/api/auth/register как есть, а Express такого роута не знает (там /auth/register). Знак ? делает слеш опциональным: работает и /api/posts, и /api/posts/, и /api/posts/1/like.

4. location /uploads/ **с** alias /var/www/uploads/ — раздача статики (аватары, фото постов, фото комментариев) напрямую через nginx, минуя сервисы. uploads-data — общий volume для auth, post, comment и nginx, поэтому все файлы лежат в одной папке.
5. Внутренние эндпоинты /internal/* не проксируются — они доступны только внутри docker-сети, что защищает их от внешних вызовов.

7. Запуск и проверка

```
cd ~/services
docker compose up -d –build
```

Все 6 контейнеров стартовали:

```
NAME            IMAGE                    STATUS
auth-service    services-auth-service    Up
comment-service services-comment-service Up
nginx           nginx:1.25-alpine        Up (0.0.0.0:80->80/tcp)
post-service    services-post-service    Up
rabbitmq        rabbitmq:3-management    Up (healthy)
social-service  services-social-service  Up
```

8. Сквозное тестирование через Nginx (порт 80)

Все запросы выполняются снаружи, через http://localhost, без указания портов сервисов:

```
# Логин
curl -X POST http://localhost/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"nginx_user2@example.com","password":"123456"}'
# → {"token":"eyJ...","user":{"id":1,"username":"nginx_user2",...}}

# Создание поста
curl -X POST http://localhost/api/posts \
  -H "Authorization: Bearer $TOKEN" \
  -F "photo=@/mnt/c/Users/user/Documents/IMG_8607_fujifilm_xtra_400_spektr.jpg" \
  -F "description=Пост через Nginx"
```

```
# → {"id":1,"originalPhotoPath":"/uploads/posts/original_1_1789806785767.jpg",...}

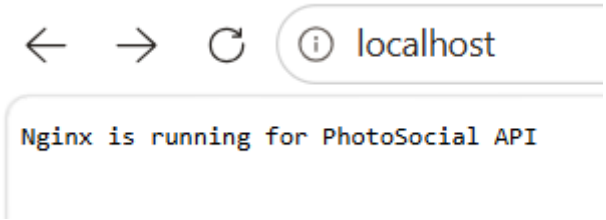
# Лайк
curl -X POST http://localhost/api/posts/1/like -H "Authorization: Bearer $TOKEN"
# → {"message":"Liked"}

# Комментарий
curl -X POST http://localhost/api/comments/posts/1/comments \
  -H "Authorization: Bearer $TOKEN" -F "text=Комментарий через Nginx"
# → {"id":1,"text":"Комментарий через Nginx",...}

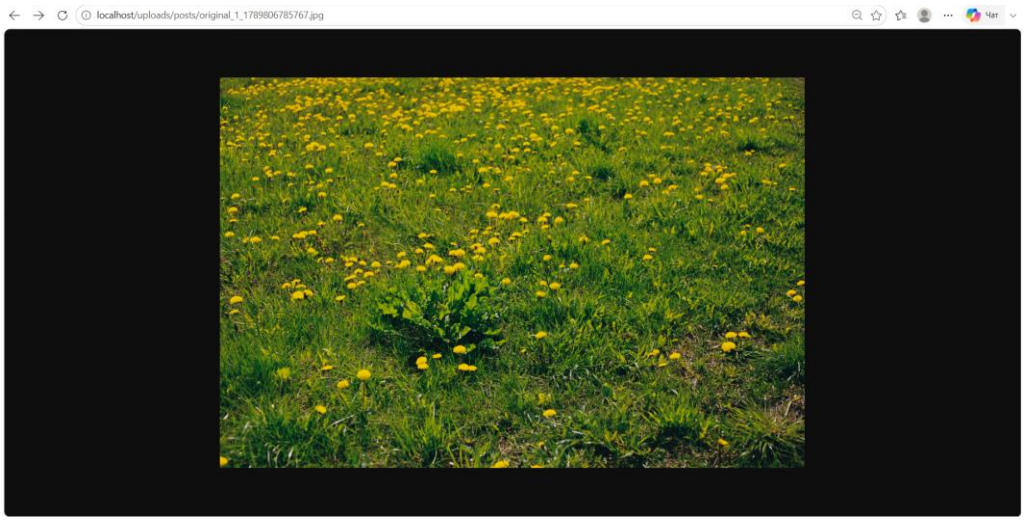
# Уведомления
curl http://localhost/api/social/notifications -H "Authorization: Bearer $TOKEN"
# → []
```

9. Проверка в браузере

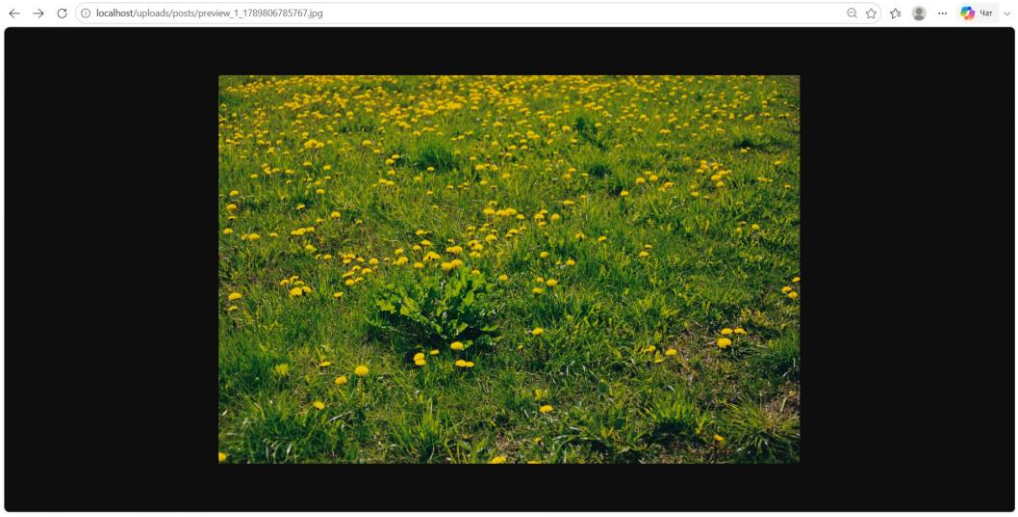
9.1. **Корень Nginx** — http://localhost/ возвращает Nginx is running for PhotoSocial API. Скриншот подтверждает работу.



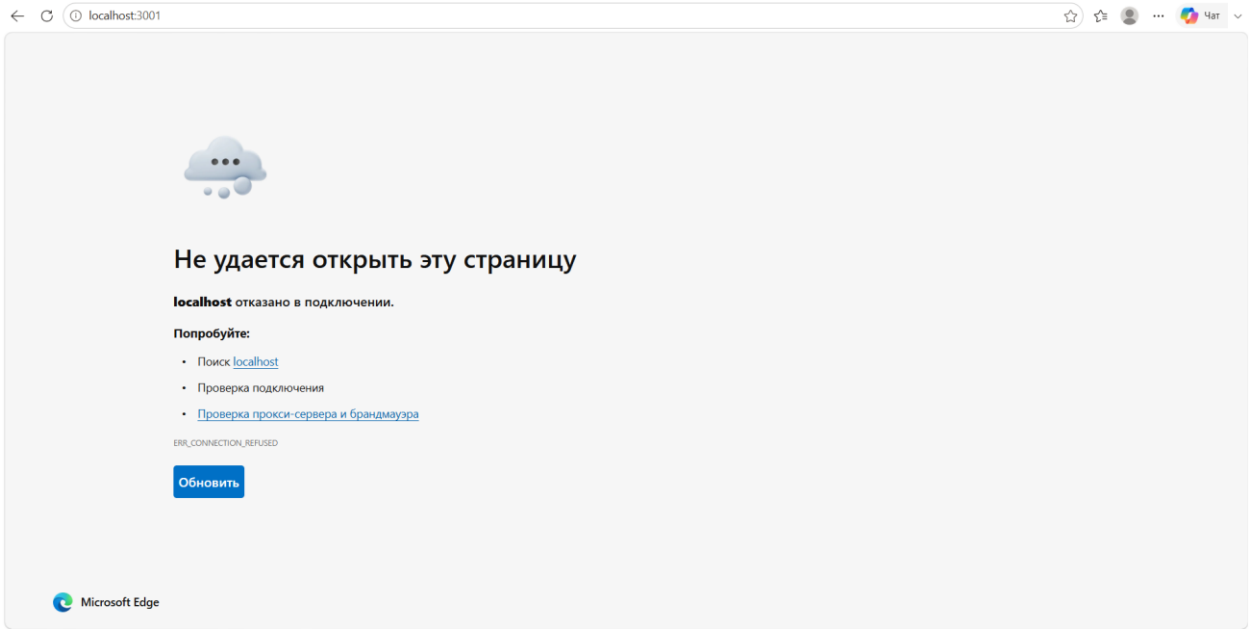
9.2. **Оригинал фото** — http://localhost/uploads/posts/original_1_1789806785767.jpg открывается в браузере. Фото в высоком разрешении (клеверное поле, ~4 МБ). Скриншот показывает, что nginx корректно раздаёт статику из /var/www/uploads/.



9.3. **Превью фото** — http://localhost/uploads/posts/preview_1_1789806785767.jpg открывается. Размер меньше (ширина 1200px), что подтверждает: sharp сгенерировал превью, nginx его отдаёт.



9.4. **Изоляция внутренних портов** — http://localhost:3001/ возвращает ERR_CONNECTION_REFUSED. Это доказывает, что auth-service не доступен снаружи напрямую — только через nginx. Все внешние запросы проходят через порт 80.



Вывод

В ходе выполнения ЛР4 микросервисное приложение фото-соцсети развёрнуто на Linux-сервере (WSL2 + Ubuntu 22.04, функционально эквивалентном удалённому VPS):

- Установлен **Docker Engine** с плагином Compose из официального репозитория.
- Настроены DNS и сеть для стабильной работы Docker в WSL2.
- Проект скопирован в Linux-файловую систему.
- Dockerfile переработаны на node:20-alpine с системным vips-dev и py3-setuptools — это решило проблемы со сборкой sharp (таймаут GitHub и отсутствие distutils).
- В docker-compose.yml добавлен сервис **nginx**, из сервисов убраны публичные порты.
- Настроен **обратный прокси** на nginx с использованием resolver 127.0.0.11 и rewrite для корректной маршрутизации путей /api/* и отдачи статики /uploads/*.

Сквозной сценарий (логин → создание поста → лайк → комментарий → уведомления) успешно пройден через порт 80 без указания портов отдельных сервисов. Все файлы (оригиналы и превью фото) доступны через nginx. Внутренние порты 3001–3004 изолированы от внешнего доступа.